

Advancing Sustainable Energy: Analyzing Stochastic Algorithms and Q-Learning for Wind Farm Layout Optimization

Mohamed Elsheikh¹, Amr Yonis¹, Jessica Magdy^{2,3} and Omar M. Shehata^{2,3}

¹ Computer Science Department, Faculty of Media Engineering and Technology, German University in Cairo, Egypt

² Mechatronics Engineering Department, Faculty of Engineering and Materials Science, German University in Cairo, Egypt

³ Multi-Robot Systems (MRS) Research Group, Cairo, Egypt

{mohamed.elsheikh, amr.younis}@student.guc.edu.eg, {jessica.gergis, omar.mohamad}@guc.edu.eg

Abstract—This paper presents a comprehensive study on the optimization of wind farm layouts, a critical aspect of enhancing the efficiency and effectiveness of wind energy production. We delve into various optimization techniques, including Simulated Annealing, Genetic Algorithm, Particle Swarm Optimization, Moth Flame Optimization, and Deep Q-Learning.

Each optimization technique is explored through a series of case studies, where different configurations and parameters are applied to assess their effectiveness in optimizing wind farm layouts. These techniques are evaluated based on criteria such as computational complexity, optimality of solutions, and repeatability of results. The paper's findings offer insightful comparisons and analyses of each technique, providing an understanding of their respective strengths and limitations in the context of wind farm layout optimization.

Index Terms—Sustainable Energy, Wind Farm Layout Optimization, Meta-Heuristic, Stochastic Algorithms, Multi-Cooperative Systems, Simulated Annealing, Genetic Algorithm, Particle Swarm, Moth Flame, Reinforcement Learning, Q-Learning

I. INTRODUCTION

IN this paper, we explore Wind Farm Layout Optimization (WFLO) to improve wind energy production efficiency, crucial for sustainable energy solutions. WFLO strategically places turbines to maximize energy output and minimize costs, addressing climate change challenges. The primary goal is to minimize the interference of wake effects, ensuring each turbine operates efficiently. We evaluate optimization techniques such as Simulated Annealing, Genetic Algorithm, Particle Swarm Optimization, Moth Flame Optimization, and Deep Q-Learning to design effective wind farm layouts. Each of these optimization algorithms offers a unique approach to solving the problem, and our analysis will shed light on their comparative strengths and weaknesses. This paper is structured to provide a comprehensive literature review, a detailed problem formulation, an in-depth analysis of each technique, and suggestions for future research.

II. LITERATURE REVIEW

WFLO optimizes wind turbine placement to minimize wake losses and maintenance costs while maximizing energy yield. In a comparative study, twelve meta-heuristic algorithms,

including GA (Genetic Algorithm), PSO (Particle Swarm Optimization), and DE (Differential Evolution), were analyzed for their adaptability and solution quality [1]. A study on Simulated Annealing's adaptability and superiority in certain aspects compared to common algorithms was conducted, lacking comprehensive evaluation across problem characteristics [2]. In a multi-objective genetic algorithm approach, optimization of wind turbine placement in a wind farm was demonstrated, showcasing the applicability of genetic algorithms to WFLO [3]. Discrete PSO extended the algorithm to discrete optimization problems, demonstrating feasibility and parameter tuning beyond traditional PSO [4]. PSO was utilized for turbine location planning, calculating annual power generation but with limited solution space and neglect of wake effects [5].

Consideration of terrain and wake behavior in wind farm optimization led to improved Levelized Production Cost (LPC) for uneven onshore wind farms. Nevertheless, further exploration in larger and more complex wind farm scenarios is still needed [6]. Moth Flame Optimization balances exploration and exploitation effectively, outperforming other algorithms [7]. Reinforcement learning combined with multi-objective DE for WFLO, adapting DE parameters through Reinforcement Learning (RL) for exploration and exploitation balance [8].

III. METHODOLOGY

A. Problem Overview

WFLO aims to arrange wind turbines strategically to maximize energy production while considering factors like turbine interference, wind variations, and environmental constraints. The goal is to design an efficient layout that minimizes wake effects and overall cost. The crucial aspect of designing an efficient layout lies in minimizing wake effects, where turbulence and reduced wind speeds in the wake of a turbine can significantly decrease the energy output of downstream turbines [9]. Due to the large number of possible solutions, conventional numerical methods are impractical, and meta-heuristic algorithms are employed to find optimal solutions. For example, even for a low number of WTs (< 30), there exists over 10^{44} unique solutions [10].

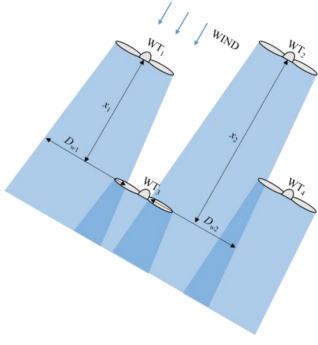


Fig. 1: Illustration of the wake effect [2]

B. Problem Description

In this scenario, we focus on optimizing the layout in an $n \times m$ grid, considering 36 wind directions with constant speed to create a versatile layout. Each wind direction will be given a weight to simulate its frequency. A higher frequency means that this particular wind direction occurs more often and thus is more important for the optimization. For our modelling of the wake effect shown in Fig. 1, we will be using a modified single wake model presented by [11], which is a variation of the model in [9], due to its simplicity and relative accuracy.

1) *Objective Function*: For the derivation of the objective function, we have 2 aspects that need to be optimized: Total power generated, and cost of procurement and maintenance. We will be using the single wake model for power calculation.

The single wake model simplifies wind wake dynamics by assuming a turbulent wake behind the rotor. The wake's size increases linearly with downstream distance (X), with uniform radial wind speed inside. It's derived from the conservation of mass, considering parameters V_1 (wind speed behind rotor), r_r (rotor radius), r_w (downstream wake radius), V_0 (freestream wind speed), and wake wind speed at distance X (V) as shown in (1).

$$\pi r_r^2 V_1 + \pi (r_w^2 - r_r^2) V_0 = \pi r_w^2 V \quad (1)$$

The velocity (V_1) at a downstream distance X can be calculated using (2):

$$\frac{V_1}{V_0} = 1 - \left(1 - \frac{1}{3} \times \frac{V}{V_0}\right) \left(\frac{r_r}{r_r + \alpha X}\right)^2 \quad (2)$$

where α is the wake decay constant. 0.075 is the recommended value for onshore wind farms [12]. The power of the wind farm at a specific direction W is given by (3).

$$P_W = \sum_i^{N_{total}} 0.3 V_i^3 \quad (3)$$

where V_i is the velocity at the i th-turbine and N_{total} is the total number of turbines. The total average power is calculated as the weighted sum of all the powers with the weights being the frequency of each wind direction shown in (4).

$$\text{Power}_{total} = \sum_{i=1}^{N_D} P_{Wi} \times W_{Di} \quad (4)$$

where N_D is the number of wind direction configurations under consideration, P_{Wi} is the power at the i th wind configuration, and W_{Di} is the weight of the i th direction.

For the calculation of the cost of procurement and maintenance, a model was presented that captures this relation according to the number of wind turbines being placed given by (5) [11].

$$\text{Cost}_P = N_{total} \left(\frac{2}{3} + \frac{1}{3} e^{-0.00174 N_{total}^2} \right) \quad (5)$$

This model suggests that up to a certain point, buying more turbines is more cost effective due to bulk discounts and hence the exponential term. Finally, the total objective function to be minimized is the aggregation of all the previously mentioned components (6).

$$\text{Objective Function} = \frac{\text{Cost}_P}{\text{Power}_{total}} \quad (6)$$

2) *Decision Variables*: There are two decision variables in our problem that directly affect the fitness computation:

- N_{total} : An integer value representing the total number of wind turbines to be erected in the farm.
- Turbine Layout: A 1-dimensional array M representing the turbines placed. Each element is a pair representing the cell currently being occupied by a turbine.

3) *Constraints*: We have identified five key constraints for our layout optimization problem, closely mirroring real-world scenarios:

- Grid Dimensions: Represented by two integers (n, m), indicating the size of the grid. The total number of cells available is $m \times n$. Turbine coordinates (x_t, y_t) must fall within this grid.
- Spacing Distance: An integer indicating the minimum required distance between turbines in all 8 directions. This is crucial for operational and safety reasons. We've set this spacing to 3 cells, ensuring adequate separation. For an array M representing the layout with pairs $M_{i,j}$, the spacing constraint is represented by (7)

$$[M_{i,j}] \Rightarrow [\neg M_{i\pm 3,j} \wedge \neg M_{i,j\pm 3} \wedge \neg M_{i\pm 3,j\pm 3}] \quad (7)$$

where the negation of $M_{i,j}$ means the pair cannot exist in the array.

- Dead Zones: An array of pairs representing cells where turbines cannot be placed. This simulates obstacles such as topographical features. This is defined as (8)

$$\text{For every element } Z \text{ in } D: \neg M_{Z_y, Z_x} \quad (8)$$

where D is the array of dead zones.

- Maximum Turbines: An integer specifying the maximum allowable number of turbines. This is constrained by grid size, spacing distance, and dead zones.

- **Minimum Operational Power Coefficient $MOPC$:** A factor that ensures the power generated at any given wind direction is not less than the average generated power across all directions multiplied by this coefficient. This constraint is described mathematically as in (9).

$$\min_{\text{wind directions}} (\text{power}) > \text{power}_{\text{avg}} \times MOPC \quad (9)$$

C. Algorithms

The implementation details for each algorithm are as follows:

1) *Simulated Annealing (SA)*: is an optimization technique inspired by metallurgical annealing. It involves gradually cooling a system to reduce defects and find better configurations.

- **Neighbor Solution Generation:** Neighbor solutions are generated by adding, removing, or moving wind turbines while respecting grid constraints.
- **Solution Evaluation and Acceptance Criterion:** Each new solution is evaluated, and if it improves the objective function value, it is accepted. Otherwise, acceptance is based on a probability calculation derived from the Boltzmann-Gibbs distribution.
- **Cooling and Convergence:** The temperature is gradually reduced (from 500 to 0) with a linear factor of 1 or a geometric factor of 0.95. This reduces the acceptance rate of worse solutions and drives convergence towards an optimal or near-optimal solution.

2) *Genetic Algorithm (GA)*: mimics natural selection to optimize solutions for complex problems. A new Generation of chromosomes (50 chromosomes in our case), representing potential solutions, is generated from the previous generation through three processes: elitism, crossover, and mutation.

- **Elitism:** Select the fittest 10% individuals from the current population to be persisting elite members.
- **Crossover:** Pair selected individuals and create offspring by combining their genetic makeup. Selection of (80%) of chromosomes occurs by rank selection. A random selection between Uniform Crossover and One Point Crossover was used. We also tested the latter on its own.
- **Mutation:** is performed to introduce small random changes to 10% of the chromosomes. We simulate bit flipping on a random number of genes by either removing, adding or moving a turbine.

3) *Particle Swarm Optimization (PSO)*: draws inspiration from bird social behavior in its optimization. Representing potential solutions as particles in a multidimensional space, PSO adjusts their positions using historical bests and collective knowledge. Each population consists of 50 particles.

- **Update Personal Best:** Each particle maintains the best solution it has encountered so far.
- **Update Global Best:** Identify the global best position among multiple particles. This position represents the overall best solution found by any particle in its cluster.

- **Particle Movement:** Update the velocity and position of each particle using a combination of its current velocity, personal best position, and the global best position. This movement is influenced by inertia, individual cognitive factors, and collective social factors according to (10).

$$\mathbf{v}_i^{t+1} = w \cdot \mathbf{v}_i^t + c_1 \cdot r_1 \cdot (\mathbf{p}_i^t - \mathbf{x}_i^t) + c_2 \cdot r_2 \cdot (\mathbf{g}^t - \mathbf{x}_i^t) \quad (10)$$

where for particle i , $\mathbf{v}_i^{t+1}$ is its new velocity at step $t+1$, $\mathbf{x}_i^t$ is its position at step t , $\mathbf{p}_i^t$ is its personal best position at step t , $\mathbf{g}^t$ is the global best position at step t , w is the weight of inertia (0.792), c_1 and c_2 are cognitive and social factors (both 1.4944), r_1 and r_2 are 2 random numbers from 0 to 1. Both star and cluster topologies were tested for particle neighborhood calculations.

Since PSO operates in a continuous space, we use a custom made discrete PSO by redefining arithmetic operators [4]. For a solution to get close to another solution, a two-step process is implemented. First, the existing turbines in the first solution need to approach their respective counterparts in the second solution. Second, it is essential for the number of turbines to be identical in both solutions. In this case, the number of wind turbines is treated as an additional variable in each solution in addition to the wind turbine positions. To implement this, the velocity vector is defined to include vectors for moving turbines and special operators designed to add or remove turbines at specific positions, or randomly over time.

4) *Moth Flame Optimization (MFO)*: is a nature-inspired algorithm based on the behavior of moths attracted to flames. Representing potential solutions as moths in a multidimensional space, MFO simulates their movements toward a flame, with flame intensity indicating solution quality. Moth movement is influenced by transverse orientation, simulating the behavior of real moths in the presence of light [7]. Each population consists of 50 moths.

- **Update Flames:** Adjust the flames based on the fitness of the moths. Higher fitness values correspond to higher flames in the ordering. Flames are always sorted so that moths can be mapped to the flames.
- **Moth Movement:** Simulate the movement of moths toward the flame based on their current positions as in (11).

$$S(M_i, F_j) = D_i \cdot e^{bt} \cdot \cos(2\pi t) + F_j \quad (11)$$

where D_i is the distance of the i th moth for the j th flame, b is the spiral size coefficient (0.3), and t is a random number in the range $[-1, 1]$. The distance D_i is calculated as in (12).

$$D_i = |F_j - M_i| \quad (12)$$

Due to the continuous nature of the MFO, we calculate the position of each turbine then discretize the values. We also calculate the delta in wind turbine numbers between

the moth and the flame using a discrete version of (11). If the moth has more turbines then the value obtained from the formula is the amount of turbines that are randomly removed. The same for the opposite case except turbines are added randomly instead.

The lower bound of the value t is adaptively decreased to -2 based on the range decay type (linear, geometric) to incentivize exploitation at later iterations. With the number of flames decreasing every iteration, a mapping was devised to divide all the moths into evenly distributed moths across the flames until there is only 1 flame left.

D. Q-Learning

Q-Learning is a classic RL algorithm that focuses on learning the value of taking specific actions in certain states. The Q-value represents the expected cumulative reward an agent can obtain by taking a particular action from a given state and following a specific policy [13].

1) *Environment*: The 3 main components are:

- **State**: We represent the environment as an $n \times m$ grid, updating it according to the agent's actions. Resetting occurs after a set number of training iterations. The grid cells encompass constraints, with values: 0 for available turbine placement, 1 for cells with turbines, 2 for non-turbine cells due to spacing constraints, and 3 for dead cells.
- **Action**: The agent can add or remove turbines while respecting constraints. To prevent repetitive actions, cells where turbines were added and removed are marked as dead cells, avoiding addition and removal at the same location.
- **Reward**: The agent's reward is based on the change in the objective function value defined as (13).

$$\text{Reward} = (\text{Old Fitness} - \text{New Fitness}) \times 10000 \quad (13)$$

2) *Configuration*:

- **Model Architecture**:
 - Input Layer Neurons: 24
 - Hidden Layers Neurons (each): 24
 - Output Layer Neurons (action size): $m \times n \times 2$
- **Exploration-Exploitation Strategy**:
 - Initial Exploration Rate (ϵ): 1.0
 - Minimum Exploration Rate: 0.01
 - Exploration Rate Decay: 0.99
- **Training Process**:
 - Number of Training Episodes: 150
 - Learning Rate (Adam Optimizer): 0.001
 - Batch Size: 32
 - Steps per Episode: 300
 - Discount Factor (γ): 0.95
 - Replay Memory Maximum Capacity: 2000 samples

IV. RESULTS AND DISCUSSION

A. Results

Due to the stochastic nature of meta-heuristic algorithms, it will prove beneficial to not just study individual instances

of algorithm performance as shown in Fig. 2-Fig. 3, but also average results over multiple executions.

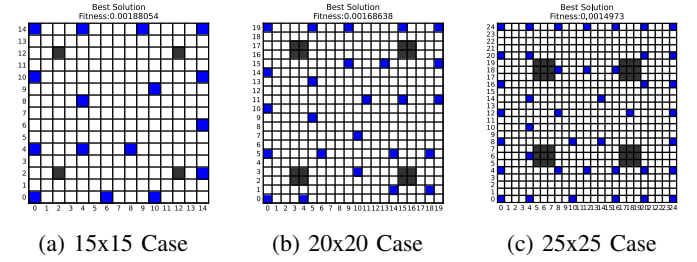


Fig. 2: Example of Solutions Achieved by GA for 3 grid sizes. Blue cells are turbines and grey cells are dead cells

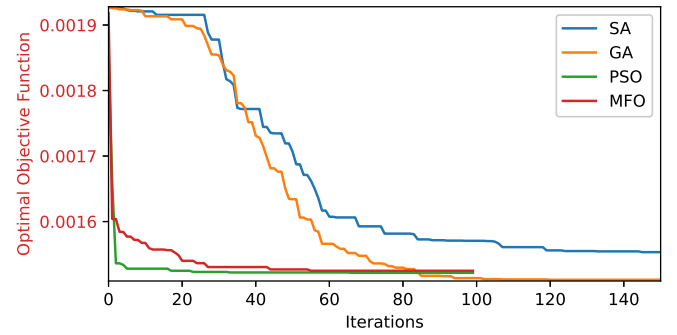


Fig. 3: Comparison of the Algorithms' Different Convergence Patterns in the 25x25 Case. All algorithms halt at specific checkpoints when reaching convergence.

We selected the highest performing configuration for each algorithm in each problem size, ran it 20 times, and recorded our metrics in table I. Additionally, the results of our q-learning experiments are in table II. Best values are highlighted in bold. The code is available on GitHub [14]. We implemented the problem in a modular fashion where each parameter can be edited including problem size with various test cases for each algorithm.

B. Discussion

1) *Run Time and Efficiency*: SA showed superior computational efficiency with shorter average run times compared to GA. PSO converged faster than GA, while MFO offered comparable performance with slightly faster run times. Larger grid sizes led to increased run times for all algorithms.

2) *Optimality and Effectiveness*: Despite longer run times, GA outperformed others in terms of optimality. PSO and MFO presented competitive solutions with faster convergence. MFO provided a balance between run time and performance.

3) *Repeatability and Consistency*: SA exhibited higher repeatability with lower fitness value deviations compared to GA. PSO showed high stability overall, while MFO maintained a consistent minimum fitness level.

TABLE I: Fitness and Run Time Statistics Categorized by Problem Size

Algorithm	Problem Size	Configuration	Best Fitness	Worst Fitness	Average Fitness	Std. Dev. of Fitness	Coefficient of Variation	Avg. Run Time (m)
SA	(15x15)	Linear	0.001926	0.001927	0.001927	0.0000004	0.00022	0.41
GA	(15x15)	1-Crossover	0.001870	0.001927	0.001888	0.0001131	0.06327	12.34
PSO	(15x15)	Star Top.	0.001871	0.001927	0.001892	0.0000169	0.00895	5.73
MFO	(15x15)	Linear	0.001871	0.001927	0.001895	0.0000196	0.01034	12.92
SA	(20x20)	Geometric	0.001760	0.001928	0.001870	0.0000596	0.03184	0.79
GA	(20x20)	2-Crossover	0.001671	0.001920	0.001725	0.0000716	0.04154	11.01
PSO	(20x20)	Star Top.	0.001678	0.001695	0.001687	0.0000040	0.00238	23.18
MFO	(20x20)	Linear	0.001676	0.001694	0.001683	0.0000045	0.00265	36.53
SA	(25x25)	Geometric	0.001508	0.001528	0.001516	0.0000051	0.00337	47.38
GA	(25x25)	2-Crossover	0.001495	0.001536	0.001508	0.0000089	0.00591	115.97
PSO	(25x25)	Star Top.	0.001514	0.001528	0.001522	0.0000040	0.00260	62.16
MFO	(25x25)	Geometric	0.001512	0.001527	0.001520	0.0000044	0.00290	53.25

TABLE II: Fitness and Reward Statistics For the Deep-Q Learning Model

Problem Size	Fitness Value	Maximum Reward	Minimum Reward	Average Reward
(15x15)	0.001914	151	0	87
(20x20)	0.001702	212	0	157
(25x25)	0.001541	314	51	232

4) *Deep-Q Learning-Based Optimization Analysis:* The DQN (Deep Q-Network) method consistently showed progress in learning across different problem sizes, as evidenced by the cumulative rewards indicating policy improvement over time. However, there were instances of reward fluctuations and instability in the learning process, indicating potential issues such as overfitting or the necessity for fine-tuning hyperparameters. Although the DQN method achieved competitive fitness values, it fell short compared to other algorithms in terms of solution quality.

V. CONCLUSION

Our analysis outlines the strengths and weaknesses of various algorithms for WFLO. When prioritizing computational efficiency, MFO offers speed with competitive optimality. GA provides highly optimal solutions but with longer runtime. SA, while efficient in runtime, may sacrifice optimality. The choice of algorithm should align with requirements, considering factors like optimality, efficiency, and repeatability.

Considering our findings, we suggest the following future research directions:

- 1) **Hybrid Models:** Develop hybrid models combining strengths of different algorithms. For example, integrating MFO with GA could enhance solution robustness.
- 2) **ML Integration:** Incorporate machine learning to predict turbine setup objective function values and improve algorithm performance.
- 3) **Scalability Studies:** Assess optimization techniques' applicability to larger problem sizes. This is especially relevant due to the computational limitations faced.

REFERENCES

- [1] T. Kunakote, N. Sabangban, S. Kumar, G. G. Tejani, N. Panagant, N. Pholdee, S. Bureerat, and A. R. Yildiz, "Comparative performance of twelve metaheuristics for wind farm layout optimisation," *Archives of Computational Methods in Engineering*, pp. 1–14, 2022.
- [2] K. Yang and K. Cho, "Simulated annealing algorithm for wind farm layout optimization: A benchmark study," *Energies*, vol. 12, no. 23, p. 4403, 2019.
- [3] M. A. Hassoune, F. Lahlou, A. Addaim, and A. A. Madi, "Wind farm layout optimization using real coded multi-population genetic algorithm," in *2019 International Conference on Wireless Technologies, Embedded and Intelligent Systems (WITS)*. IEEE, 2019, pp. 1–5.
- [4] M. Clerc, "Discrete particle swarm optimization, illustrated by the traveling salesman problem," vol. 47, 01 2004.
- [5] İ. Çelik, C. Yildiz, and M. Şekkel, "Wind power plant layout optimization using particle swarm optimization," *Turkish Journal of Engineering*, vol. 5, no. 2, pp. 89–94, 2021.
- [6] X. Wu, W. Hu, Q. Huang, C. Chen, Z. Chen, and F. Blaabjerg, "Optimized placement of onshore wind farms considering topography," *Energies*, vol. 12, no. 15, p. 2944, 2019.
- [7] S. Mirjalili, "Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm," *School of Information and Communication Technology, Griffith University, Nathan Campus, Brisbane, QLD 4111, Australia and Queensland Institute of Business and Technology, Mt Gravatt, Brisbane, QLD 4122, Australia*, pp. 227–249, 2015.
- [8] X. Yu and Y. Lu, "Reinforcement learning-based multi-objective differential evolution for wind farm layout optimization," *Energy*, vol. 284, p. 129300, 2023.
- [9] N. O. Jensen, *A note on wind generator interaction*. Citeseer, 1983, vol. 2411.
- [10] G. Gualtieri, "A novel method for wind farm layout optimization based on wind turbine selection," *Energy Conversion and Management*, vol. 193, pp. 106–123, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0196890419304935>
- [11] T. A. Qureshi and V. Warudkar, "Wind farm layout optimization through optimal wind turbine placement using a hybrid particle swarm optimization and genetic algorithm," *Environmental Science and Pollution Research*, pp. 1–17, 2023.
- [12] J. Kollwitz, "Defining the wake decay constant as a function of turbulence intensity to model wake losses in onshore wind farms," 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:53723873>
- [13] C. J. C. H. Watkins, "Learning from delayed rewards," Ph.D. dissertation, King's College, Cambridge, UK, May 1989. [Online]. Available: http://www.cs.rhul.ac.uk/~chrisw/new_thesis.pdf
- [14] M. Elsheikh and A. Yonis, "WFLO-Optimization." [Online]. Available: <https://github.com/IrrationalInteger/WFLO-Optimization>